Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Multi Agents System with AutoGen (C1034)

Course Code: C1034

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	11 August 2026	Initial release aligned across slide deck, Learner Guide, Lesson Plan, and eight connected labs.	Tertiary Infotech Trainer

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

Introduction **4**
Course Learning Outcomes **4**
Before You Start — Preparation **4**
Topic 01 — Introduction to AutoGen and CrewAI **5**
Lab 1 — Verify AutoGen and Run a First Agent 7
Lab 2 — Add a Typed Ticket Lookup Tool 9
Recap — Introduction to AutoGen and CrewAI 11
Topic 02 — Retrieval-Augmented Generation (RAG) **12**
Lab 3 — Prepare a Source-Traceable Knowledge Index 13
Lab 4 — Evaluate Retrieval with Stable Questions 15
Lab 5 — Build a Grounded RAG Agent 17
Recap — Retrieval-Augmented Generation (RAG) 20
Topic 03 — Implementing a Multi-Agent AI Workflow **20**
Lab 6 — Design Roles and Run a Round-Robin Team 22
Lab 7 — Route Specialists with SelectorGroupChat 24
Lab 8 — Integrate and Verify the OpsDesk Workflow 27
Recap — Implementing a Multi-Agent AI Workflow 30
Wrap-Up **30**
Next Steps **31**
Glossary **31**

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Introduction

This Learner Guide accompanies Multi Agents System with AutoGen (C1034), a two-day, 15-hour beginner course for developers, data practitioners, solution architects and AI automation professionals. It begins with the difference between a language model, an agent and a multi-agent system, then makes the AgentChat execution model visible through reproducible messages, events and stop reasons.

All eight labs extend one synthetic OpsDesk scenario. You will run a first AssistantAgent, add a typed read-only ticket tool, prepare and evaluate source-labelled knowledge, build a grounded RAG agent, compare round-robin and selector-based teams, and finish with a regression and operational readiness record. The examples target AutoGen AgentChat 0.7.5 as available on 11 August 2026 and keep paid live model calls optional.

Course Learning Outcomes

- LO1: Explain LLM agents, the AgentChat execution model, and the practical differences between AutoGen and CrewAI abstractions.
- LO2: Configure and run an AutoGen AssistantAgent with a model client, clear messages, a typed read-only tool, and inspectable event evidence.
- LO3: Design a Retrieval-Augmented Generation pipeline with source-labelled chunks, transparent ranking, metadata, context limits, and stable retrieval questions.
- LO4: Build and verify a grounded RAG agent that cites retrieved evidence, treats documents as untrusted data, and handles insufficient evidence honestly.
- LO5: Select and implement appropriate multi-agent patterns with specialist roles, explicit handoffs, shared context, and bounded speaker routing.
- LO6: Apply termination, state management, guardrails, observability, regression checks, ownership, and recovery controls to an integrated AutoGen workflow.

Before You Start — Preparation

What you need

- A Windows, macOS or Linux laptop with Python 3.10 or later, Git and a modern code editor.
- Internet access for the initial package installation; the guided verification path then runs offline.
- The course repository cloned locally and a virtual environment created for its dependencies.
- Optional only: an OpenAI API key for separately labelled live comparisons. Store it in a local .env file or environment variable.
- Only the supplied synthetic incidents and handbook content; no customer, employee or production data is needed.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Verify your setup

From the repository root, activate the virtual environment, install the pinned starter requirements and editable starter package, then run the setup report. The expected AutoGen packages are version 0.7.5 and the final status is ready.

```
python --version

python -m pip install -r starter/requirements.txt

python -m pip install -e starter

python -m opsdesk.cli setup
```

Conventions used in every lab

- Run commands from the repository root after installing starter in editable mode, unless a step explicitly changes directory.
- Use --offline for the required reproducible evidence. Optional live output is compared separately and never replaces the offline checkpoint.
- Replace placeholders such as <OPENAI_API_KEY> locally; never commit a real secret or paste it into a prompt, screenshot or evidence file.
- Use the supplied INC-1001 to INC-1003 incidents and POL-/PROC- source identifiers only.
- Inspect .course_state after every run; it holds the JSON evidence needed by later labs and is intentionally ignored by Git.

Topic 01 — Introduction to AutoGen and CrewAI

LLMs and agents · AutoGen layers · AgentChat lifecycle · AssistantAgent · tools · messages · state · framework choice

Key concepts

LLM, agent and multi-agent system

An LLM predicts outputs from context. An agent wraps a model with instructions, tools, state and an execution loop. A multi-agent system adds role boundaries and communication, which increases coordination work as well as capability.

The agent loop

A task becomes messages; the model chooses a response or tool call; the application executes approved tools; the result returns as another event; and an explicit rule decides whether to continue or stop.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

AutoGen layers

Studio supports visual prototyping, AgentChat provides high-level conversational agents and teams, Core exposes event-driven primitives, and Extensions connect model clients, memory, runtimes and tools.

AgentChat contract

AgentChat agents have unique names and useful descriptions, accept tasks through run or run_stream, emit typed messages and events, and retain state until reset. The returned TaskResult contains the run history and stop reason.

AssistantAgent

AssistantAgent is a general preset for prototyping and education. Its contract combines a model client, system message, optional tools, model context and bounded tool iterations; specialised production agents may use narrower custom implementations.

Messages and events

User and agent text are chat messages. Tool requests, tool results and streaming chunks are events. Operators examine both because a fluent final answer cannot prove that the correct route or evidence was used.

Model clients

A model client separates orchestration from a provider. The course uses ReplayChatCompletionClient for reproducible offline evidence and an OpenAI client only for optional live comparison through environment-held credentials.

Typed tools

A function name, docstring, parameter types and return shape become the tool schema. Clear read-only contracts reduce ambiguous calls and make argument validation, error handling and evidence capture testable.

State and reset

Agents and teams retain conversation state after a run. Continue only when the next task belongs to the same thread; reset before an unrelated task so stale context does not silently influence a result.

Bounded autonomy

Every loop needs a useful completion condition plus a hard message, tool-iteration, token or time limit. A limit is not a success signal; the stop reason must remain visible to the application.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

AutoGen and CrewAI

AutoGen AgentChat centres on typed messages, events, model clients and team presets. CrewAI organises role-based agents into crews and tasks, while Flows provide explicit event-driven control. Choose by required control, integrations and team operating model.

Worked example: OpsDesk

A synthetic service request enters one read-only assistant. The agent looks up an incident, later retrieves handbook evidence, then grows into planner, retriever, writer and reviewer roles without gaining permission to change production systems.

Lab 1 — Verify AutoGen and Run a First Agent

Learning outcome: LO1 and LO2: distinguish the AgentChat components, prepare a reproducible environment, run an AssistantAgent, and inspect messages and the TaskResult stop reason.

Goal: You prepare an isolated Python environment, verify the pinned AutoGen AgentChat and Extensions packages, and run the first OpsDesk AssistantAgent with a ReplayChatCompletionClient. You then inspect the saved message types, final text and stop reason so the execution model is observable before any live model or tool is introduced.

What you'll build

A ready environment plus .course_state/lab01-environment.json and lab01-first-agent.json containing the first reproducible AgentChat trace. (Tools: Python 3.10+, virtual environment, AutoGen AgentChat 0.7.5, AutoGen Extensions replay client, OpsDesk CLI.)

Prerequisites and time

- The C1034 repository is available locally and the current terminal is at its root.
- Python 3.10 or later and internet access for the initial package installation are available.
- No provider credential is required for the offline path.

Suggested duration: 60 minutes.

Step-by-step

1. Create the course virtual environment from the repository root. If .venv already exists, keep it and continue rather than replacing it.

```
python -m venv .venv
```

2. Activate the environment with the command for your shell. Confirm the prompt shows .venv before installing anything.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
# Windows PowerShell

.\.venv\Scripts\Activate.ps1

# macOS or Linux

source .venv/bin/activate
```

3. Install the pinned dependencies and the editable OpsDesk lab package. The editable install makes python -m opsdesk.cli work from the repository root.

```
python -m pip install -r starter/requirements.txt

python -m pip install -e starter
```

4. Generate the environment report. Check Python support, both AutoGen package versions, the two data files and the final ready status.

```
python -m opsdesk.cli setup
```

5. Inspect the saved setup evidence instead of relying only on terminal output. The package versions must both be 0.7.5.

```
python -m json.tool .course_state/lab01-environment.json

python -m pip show autogen-agentchat autogen-ext
```

6. Run the first AssistantAgent with the deterministic replay model. The output must describe the synthetic-data, citation and insufficient-evidence boundaries.

```
python -m opsdesk.cli first-agent --offline
```

7. Inspect the trace. Identify the agent name, ordered message types, final content, null team stop_reason and application-level completion_reason. A one-turn AssistantAgent run completes without a team termination condition.

```
python -m json.tool .course_state/lab01-first-agent.json
```

8. Run the focused offline check and review the bounded repository diff. Keep both JSON files as the rejoin checkpoint for Lab 2.

```
python -m pytest -q starter/tests/test_examples.py -k first_agent

git status --short
```

Test it

.course_state/lab01-environment.json must report status ready, Python support true and AutoGen package versions 0.7.5. lab01-first-agent.json must name opsdesk_assistant, include TextMessage entries, contain the synthetic-data boundary, preserve a null team stop_reason and report completion_reason single_agent_turn_complete. The focused test must be clean.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Troubleshooting

- PowerShell blocks activation. Run Set-ExecutionPolicy -Scope Process Bypass for the current terminal only, then activate .venv again.
- The setup report says a package is missing. Confirm the virtual environment is active, rerun both pip install commands, and use python -m pip so installation and execution use the same interpreter.
- python cannot import opsdesk. Run python -m pip install -e starter from the repository root and confirm starter/pyproject.toml is present.

Challenge

Run python -m opsdesk.cli first-agent --offline twice, compare the two JSON files, and explain why deterministic replay is useful for a regression baseline but cannot estimate live-model quality.

Reflection

Why does a one-turn AssistantAgent run have no team stop reason, and which application-level field should describe its successful completion?

Note: Full commands are in labs/lab-01-*.md. Use only the supplied synthetic incidents and handbook. Keep tools read-only, store optional provider credentials outside Git, and never place a secret in prompts, screenshots, commands or saved evidence.

Lab 2 — Add a Typed Ticket Lookup Tool

Learning outcome: LO2: define and execute a typed read-only tool, inspect its schema and tool events, and preserve explicit not-found behaviour without granting write authority.

Goal: You extend the OpsDesk assistant with lookup_ticket, a typed function that reads one synthetic incident by exact identifier. The offline replay client produces a real function-call request, AgentChat executes the tool, and the saved trace proves the request, result and not-found behaviour without changing any ticket.

What you'll build

A verified lookup_ticket contract plus .course_state/lab02-tool-agent.json containing ToolCallRequestEvent, ToolCallExecutionEvent and the exact synthetic result. (Tools: AssistantAgent, FunctionTool schema, FunctionCall, ToolCallExecutionEvent, ReplayChatCompletionClient, synthetic tickets.json.)

Prerequisites and time

- Lab 1 setup status is ready and its focused test is clean.
- The editable OpsDesk package is installed in the active virtual environment.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- starter/data/tickets.json still contains only the supplied INC-1001 to INC-1003 records.

Suggested duration: 55 minutes.

Step-by-step

1. Confirm the Lab 1 checkpoint and active package versions before changing the agent boundary.

```
python -m opsdesk.cli setup
```

```
python -m pytest -q starter/tests/test_examples.py -k first_agent
```

2. Inspect the supplied incidents and the lookup_ticket function. Note the exact string parameter, docstring and stable JSON statuses found and not_found.

```
python -m json.tool starter/data/tickets.json
```

```
python -c "from opsdesk.examples import lookup_ticket; import inspect;
print(inspect.signature(lookup_ticket)); print(inspect.getdoc(lookup_ticket))"
```

3. Create and execute the FunctionTool directly once. This separates schema and application behaviour from model selection.

```
python -c "import asyncio, json; from opsdesk.examples import inspect_tool_schema;
print(json.dumps(asyncio.run(inspect_tool_schema()), indent=2, default=str))"
```

4. Run the AgentChat tool path for INC-1001. The replay completion requests lookup_ticket and AutoGen emits the real tool execution events.

```
python -m opsdesk.cli tool-agent --ticket INC-1001 --offline
```

5. Inspect the saved evidence. Confirm tool_event_seen is true, the final content contains INC-1001 and no event claims that a record was changed.

```
python -m json.tool .course_state/lab02-tool-agent.json
```

6. Exercise the unknown-identifier route. It must return status not_found with the requested identifier rather than inventing a ticket.

```
python -m opsdesk.cli tool-agent --ticket INC-9999 --offline
```

```
python -m json.tool .course_state/lab02-tool-agent.json
```

7. Rerun the normal case so the checkpoint represents the shared scenario, then run the focused tool test.

```
python -m opsdesk.cli tool-agent --ticket INC-1001 --offline
```

```
python -m pytest -q starter/tests/test_examples.py -k tool_agent
```

8. Confirm the synthetic dataset has no Git change and keep the normal trace as the Lab 2 checkpoint for retrieval work.

```
git diff -- starter/data/tickets.json
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
git status --short
```

Test it

The normal trace must set tool_event_seen true and contain INC-1001. The unknown route must return not_found without an invented record, the focused test must be clean, and starter/data/tickets.json must remain unchanged.

Troubleshooting

- The trace has no ToolCallExecutionEvent. Confirm --offline is present and the installed packages are 0.7.5, then inspect _tool_call and the exact lookup_ticket function name in starter/opsdesk/examples.py.
- The tool returns not_found for INC-1001. Run from the repository root, confirm starter/data/tickets.json is present, and verify the identifier uses uppercase INC and a hyphen.
- The source dataset appears modified. Restore the supplied synthetic values from the repository history before continuing; later stable checks depend on exact identifiers and categories.

Challenge

Add a new read-only category filter in a separate branch, define its typed return contract, and write a test proving it cannot add, edit or delete a ticket.

Reflection

Which part of the tool contract prevents a fluent agent response from silently turning an unknown ticket into a fictional record?

Note: Full commands are in labs/lab-02-*.md. Use only the supplied synthetic incidents and handbook. Keep tools read-only, store optional provider credentials outside Git, and never place a secret in prompts, screenshots, commands or saved evidence.

Recap — Introduction to AutoGen and CrewAI

- You can now: LO1 and LO2: distinguish the AgentChat components, prepare a reproducible environment, run an AssistantAgent, and inspect messages and the TaskResult stop reason
- You can now: LO2: define and execute a typed read-only tool, inspect its schema and tool events, and preserve explicit not-found behaviour without granting write authority

Use the lab verification evidence to explain which parts are deterministic code, model-driven judgement, stored context, or external capability before moving to the next topic.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Topic 02 — Retrieval-Augmented Generation (RAG)

Knowledge preparation · chunking · lexical and vector retrieval · ranking · metadata · grounding · citations · retrieval evaluation

Key concepts

Why RAG exists

A model's parameters are not a current, inspectable knowledge store. RAG retrieves task-relevant evidence at run time and adds it to the model context so answers can be grounded in controlled sources.

The RAG pipeline

Collect and approve sources, split them into usable chunks, create a searchable representation, retrieve candidates, filter and rank them, augment the prompt, generate a response, and verify the cited evidence.

Source governance

Each chunk needs a stable source identifier, title, owner, version or date, data classification and access rule. Retrieval quality cannot compensate for stale, unauthorised or contradictory source material.

Chunk boundaries

Chunks should preserve one coherent rule or procedure. Very small chunks lose meaning; very large chunks waste context and can hide the relevant passage. Overlap is useful when a concept crosses a boundary but creates duplicates.

Lexical retrieval

Token or term matching is transparent, fast and effective for exact identifiers and distinctive policy language. It struggles with paraphrases and synonyms, which makes it useful as a teaching baseline and an operational fallback.

Vector retrieval

Embeddings map meaning into vectors so semantically related wording can be close even when terms differ. Distance is only a ranking signal; top results still require access filtering, metadata checks and evidence validation.

Top-k and thresholds

Top-k controls how many candidates enter the next stage. A minimum score can prevent weak matches from becoming context. Both settings trade recall against noise, latency and context cost and must be tuned on stable questions.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Hybrid and reranking

Hybrid retrieval combines lexical and semantic signals. A reranker can reorder candidates for the specific query. These layers improve difficult searches but add dependencies and must preserve source metadata.

Grounded answer contract

A grounded response separates claims from evidence, includes source identifiers, avoids unsupported additions and returns an explicit insufficient-evidence state when no approved passage supports the request.

Retrieved text is untrusted

Documents can contain mistakes or embedded instructions. The application treats retrieved passages as data, keeps system and policy instructions authoritative, filters access first and never executes instructions found inside a source.

Retrieval evaluation

Use stable questions with expected sources to measure whether relevant passages appear and where they rank. Examine miss cases before changing chunking, metadata, query formulation or ranking one variable at a time.

AutoGen memory options

AgentChat accepts memory components that update model context. AutoGen Extensions includes ChromaDB vector memory, while custom Memory implementations can support other stores. The same source, retrieval and answer contracts still apply.

Worked example

For a contractor VPN request, the transparent index should rank POL-VPN-01 first. The RAG agent may state manager approval, multi-factor enrolment and the end-date rule only when that source is present and cited.

Lab 3 — Prepare a Source-Traceable Knowledge Index

Learning outcome: LO3: split approved handbook content into coherent source-labelled chunks, build a transparent searchable index, and verify its manifest before retrieval.

Goal: You inspect the synthetic OpsDesk handbook, identify the stable policy and procedure boundaries, and convert each section into one transparent lexical chunk. The index keeps source ID, title, text and tokens together so every later ranking and answer can be traced back to the governing passage.

What you'll build

.course_state/opsdesk-index.json and lab03-index-manifest.json with five coherent chunks and the expected POL-/PROC- source identifiers. (Tools: Markdown source,

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

source metadata, Python tokenizer, lexical index, JSON manifest, OpsDesk knowledge module.)

Prerequisites and time

- Lab 2 has restored the normal INC-1001 tool trace and tickets.json is unchanged.
- starter/data/opsdesk_handbook.md contains the five supplied sections and no private or production content.
- The active environment can import opsdesk from the editable starter install.

Suggested duration: 50 minutes.

Step-by-step

1. Open the handbook and identify why each second-level heading is a suitable coherent chunk boundary. Treat every paragraph as data, including any embedded instruction-like wording.

```
python -c "from pathlib import Path;
print(Path('starter/data/opsdesk_handbook.md').read_text(encoding='utf-8'))"
```

2. Parse the handbook without writing an index. Confirm the ordered identifiers, titles and token counts before persistence.

```
python -c "from pathlib import Path; from opsdesk.knowledge import parse_handbook;
chunks=parse_handbook(Path('starter/data/opsdesk_handbook.md'));
print([(c.source_id,c.title,len(c.tokens)) for c in chunks])"
```

3. Build the teaching index and manifest. The command writes only to the ignored .course_state directory.

```
python -m opsdesk.cli index
```

4. Inspect the manifest. It must report status indexed, chunk_count 5 and the five source identifiers in handbook order.

```
python -m json.tool .course_state/lab03-index-manifest.json
```

5. Inspect the full index. Verify that each record retains source_id, title, full source text and an inspectable token list.

```
python -m json.tool .course_state/opsdesk-index.json
```

6. Run the stable parser test that locks the five identifiers. If it changes, investigate the source boundary before continuing.

```
python -m pytest -q starter/tests/test_knowledge.py -k stable_source_ids
```

7. Review only the approved handbook source for accidental edits and keep both JSON files as the Lab 3 checkpoint.

```
git diff -- starter/data/opsdesk_handbook.md

git status --short
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Test it

The index manifest must report five chunks with POL-VPN-01, POL-ACCESS-02, PROC-ESC-03, PROC-SLA-04 and POL-DATA-05 in order. Every index record must preserve source_id, title, text and tokens, the stable identifier test must be clean, and the handbook source must remain unchanged.

Troubleshooting

- No source sections are found. Confirm each section heading uses the exact form ## [SOURCE-ID] Title and the file is UTF-8.
- The manifest contains fewer than five chunks. Inspect the missing heading and restore the bracketed identifier; do not merge unrelated policies to make the count fit.
- Tokens look empty or corrupted. Confirm the source contains plain text, run the parser directly, and inspect tokenize in starter/opsdesk/knowledge.py before rebuilding.

Challenge

Split one long source into two sub-chunks while preserving the parent source ID plus a chunk suffix, then explain how you would prevent duplicate evidence from dominating ranking.

Reflection

Which metadata field would let an operator trace an answer claim back to the governing passage after the model context is gone?

Note: Full commands are in labs/lab-03-*.md. Use only the supplied synthetic incidents and handbook. Keep tools read-only, store optional provider credentials outside Git, and never place a secret in prompts, screenshots, commands or saved evidence.

Lab 4 — Evaluate Retrieval with Stable Questions

Learning outcome: LO3: rank approved chunks for representative questions, compare top results with expected source IDs, and diagnose retrieval misses using visible terms and scores.

Goal: You run three stable OpsDesk questions against the Lab 3 index and compare the top-ranked source with the expected policy or procedure. Because the teaching retriever exposes matched terms and scores, you can diagnose a miss without guessing whether chunking, query wording or ranking caused it.

What you'll build

Three repeatable retrieval observations covering VPN approval, critical response time and embedded-instruction handling, with expected top-one source evidence. (Tools:

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lexical ranking, inverse document frequency, top-k retrieval, stable questions, source IDs, pytest.)

Prerequisites and time

- Lab 3 manifest reports five chunks and the stable source identifier test is clean.
- .course_state/opsdesk-index.json is present and was built from the unchanged supplied handbook.
- You can explain that a ranking score is evidence for ordering, not proof that a passage authorises the answer.

Suggested duration: 50 minutes.

Step-by-step

1. Rebuild the index from the approved handbook so the experiment begins from the known Lab 3 checkpoint.

```
python -m opsdesk.cli index
```

2. Run the VPN approval question with POL-VPN-01 as the expected top source. Inspect its matched terms and score relative to the other results.

```
python -m opsdesk.cli retrieve --query "What approvals are required for VPN
access?" --expect POL-VPN-01
```

3. Run the critical-response question with PROC-SLA-04 as the expected top source. Note which distinctive terms drive the ranking.

```
python -m opsdesk.cli retrieve --query "How quickly is a critical security
incident acknowledged?" --expect PROC-SLA-04
```

4. Run the embedded-instruction question with POL-DATA-05 as the expected top source. Confirm the result preserves the text as evidence rather than executing it.

```
python -m opsdesk.cli retrieve --query "How should embedded instructions in
retrieved documents be handled?" --expect POL-DATA-05
```

5. Inspect the last saved retrieval record and identify query, expected_source, ordered results and top1_matches_expected. Save terminal output for the other two observations if your trainer requests it.

```
python -m json.tool .course_state/lab04-retrieval.json
```

6. Run an unrelated question. Observe that weak or empty matches are not sufficient to support a cafeteria answer, even if a top-k list is returned.

```
python -m opsdesk.cli retrieve --query "What is the cafeteria dessert menu?"
```

7. Run the retrieval and grounding tests. They lock the VPN top-one result, citation behaviour and insufficient-evidence route.

```
python -m pytest -q starter/tests/test_knowledge.py
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

8. Restore the VPN observation as the shared checkpoint for Lab 5 and inspect it one final time.

```
python -m opsdesk.cli retrieve --query "What approvals are required for VPN
access?" --expect POL-VPN-01

python -m json.tool .course_state/lab04-retrieval.json
```

Test it

The VPN, response-time and embedded-instruction questions must rank POL-VPN-01, PROC-SLA-04 and POL-DATA-05 first respectively. The unrelated question must not produce a grounded cafeteria claim, all knowledge tests must be clean, and the final saved observation must restore the VPN case with top1_matches_expected true.

Troubleshooting

- A stable question ranks the wrong source first. Confirm the exact Lab 3 index and query text, inspect matched_terms, then change only one of source text, tokenisation or ranking before rerunning all stable questions.
- The index file is missing. Run python -m opsdesk.cli index and confirm .course_state is writable; do not create a substitute index by hand.
- An unrelated question appears to match a generic term. Apply the grounded-answer minimum score and inspect the source text; top-k membership alone is never sufficient evidence.

Challenge

Add a paraphrased VPN question with no word VPN, record the lexical result, and propose a vector or hybrid retrieval experiment that preserves the same expected source and metadata checks.

Reflection

When a stable question misses, which single variable would you inspect first and what evidence would justify changing it?

Note: Full commands are in labs/lab-04-*.md. Use only the supplied synthetic incidents and handbook. Keep tools read-only, store optional provider credentials outside Git, and never place a secret in prompts, screenshots, commands or saved evidence.

Lab 5 — Build a Grounded RAG Agent

Learning outcome: LO4: connect retrieval as an AgentChat tool, require source-labelled responses, verify tool events and citations, and preserve an insufficient-evidence route.

Goal: You give a knowledge AssistantAgent access to the verified teaching retriever, require a tool call before answering and add a reflection step that turns tool evidence

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

into a concise cited response. You then compare the supported VPN request with an unrelated question and confirm that retrieval events and source IDs remain visible.

What you'll build

.course_state/lab05-rag-agent.json containing the RAG tool trajectory, grounded final text, source identifiers and stop reason for the shared VPN question. (Tools: AssistantAgent, retrieval FunctionTool, reflect_on_tool_use, max_tool_iterations, source citations, insufficient-evidence route.)

Prerequisites and time

- Lab 4 has restored the VPN retrieval observation with POL-VPN-01 ranked first.
- .course_state/opsdesk-index.json and the unchanged handbook are available.
- The AgentChat tool path from Lab 2 still emits real request and execution events.

Suggested duration: 75 minutes.

Step-by-step

1. Restore and verify the index and VPN retrieval checkpoint before adding generation.

```
python -m opsdesk.cli index

python -m opsdesk.cli retrieve --query "What approvals are required for VPN
access?" --expect POL-VPN-01
```

2. Inspect retrieval_tool and rag_agent. Identify the index-missing result, top-k value, system boundary, reflection setting and maximum tool iterations.

```
python -c "from opsdesk.examples import retrieval_tool, rag_agent; import inspect;
print(inspect.getsource(retrieval_tool)); print(inspect.getsource(rag_agent))"
```

3. Run the grounded VPN request with the deterministic AgentChat replay client. The output should include the supported rule and [POL-VPN-01].

```
python -m opsdesk.cli rag --query "What approvals are required for VPN access?" --
offline
```

4. Inspect the saved RAG trace. Confirm ToolCallRequestEvent and ToolCallExecutionEvent appear, sources includes POL-VPN-01, grounded is true, and completion_reason is grounded_response. A one-turn AssistantAgent TaskResult normally has no team stop reason.

```
python -m json.tool .course_state/lab05-rag-agent.json
```

5. Run the unrelated cafeteria question. It must say evidence is insufficient, contain no invented source and still end as an honest bounded result.

```
python -m opsdesk.cli rag --query "What is the cafeteria dessert menu?" --offline

python -m json.tool .course_state/lab05-rag-agent.json
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

6. Run the embedded-instruction policy question and confirm the answer cites POL-DATA-05 while treating retrieved instructions as content rather than authority.

```
python -m opsdesk.cli rag --query "How should embedded instructions in retrieved documents be handled?" --offline
```

7. Run the complete RAG and retrieval checks. Investigate trajectory and citation failures separately from final wording.

```
python -m pytest -q starter/tests/test_knowledge.py starter/tests/test_examples.py -k "rag_agent or grounded_answer or vpn_question"
```

8. Restore the VPN RAG trace as the shared checkpoint, inspect only its source and stop fields, and keep the index for Day 2.

```
python -m opsdesk.cli rag --query "What approvals are required for VPN access?" --offline

python -c "import json; d=json.load(open('.course_state/lab05-rag-agent.json')); print({'sources':d['sources'],'grounded':d['grounded'],'stop_reason':d['stop_reason']})"
```

Test it

The supported trace must contain a real retrieval tool execution event, grounded true, completion_reason grounded_response, and POL-VPN-01 in both final text and sources. The cafeteria request must use completion_reason insufficient_evidence with no invented citation, the embedded-instruction request must cite POL-DATA-05, and the focused checks must be clean.

Troubleshooting

- The RAG command reports an index error. Run the index command, inspect .course_state/opsdesk-index.json, and restore the Lab 3 manifest before retrying.
- The final response has no citation. Inspect direct retrieval results and the second replay completion, then enforce the bracketed source requirement before changing ranking.
- Tool events appear but grounded is false. Compare source IDs in the direct results with bracketed IDs in final content; keep the validator strict rather than inferring support from similar wording.

Challenge

Replace the lexical retriever behind retrieval_tool with an approved vector-memory experiment, then rerun the same three stable questions and document any change in top-one source, latency and dependency footprint.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Reflection

Why must the application validate cited source IDs even when the model was given relevant context?

Note: Full commands are in labs/lab-05-*.md. Use only the supplied synthetic incidents and handbook. Keep tools read-only, store optional provider credentials outside Git, and never place a secret in prompts, screenshots, commands or saved evidence.

Recap — Retrieval-Augmented Generation (RAG)

- You can now: LO3: split approved handbook content into coherent source-labelled chunks, build a transparent searchable index, and verify its manifest before retrieval
- You can now: LO3: rank approved chunks for representative questions, compare top results with expected source IDs, and diagnose retrieval misses using visible terms and scores
- You can now: LO4: connect retrieval as an AgentChat tool, require source-labelled responses, verify tool events and citations, and preserve an insufficient-evidence route

Use the lab verification evidence to explain which parts are deterministic code, model-driven judgement, stored context, or external capability before moving to the next topic.

Topic 03 — Implementing a Multi-Agent AI Workflow

Specialist roles · round robin · selector routing · handoffs · termination · state · human control · observability · release readiness

Key concepts

When multiple agents help

Split work when roles need different instructions, tools, permissions, models, ownership or quality checks. Keep one agent when a single bounded instruction and tool surface remains understandable.

Role contract

Each participant needs a unique name, capability description, task boundary, approved tools, input evidence, output format and escalation behaviour. Overlapping roles create routing ambiguity and duplicate work.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Handoff contract

A handoff carries the task, validated context, source identifiers, constraints, expected output and remaining budget. It should not expose hidden reasoning, unrelated history or broader credentials.

RoundRobinGroupChat

Round robin gives every participant a predictable turn and shared context. It fits drafting and review loops or demonstrations where the order is intentional, but wastes turns when some roles are not needed.

SelectorGroupChat

A selector chooses the next speaker from names, descriptions and conversation context. It supports dynamic workflows but adds a routing decision that needs clear descriptions, candidate constraints, telemetry and fallback behaviour.

Custom selection

A selector function can enforce deterministic state transitions and return None to defer to model selection. Custom functions are useful for policy routes but require separate serialization and recovery design.

Other team patterns

Swarm uses handoff messages for decentralised transitions, Magentic-One targets open-ended generalist tasks, and GraphFlow expresses directed workflows. Pattern choice should follow control needs rather than novelty.

Termination

Semantic conditions such as an approval phrase or handoff should be combined with a hard maximum message, token or time condition. The team returns a stop reason that the application must interpret separately from answer content.

State lifecycle

Team and participant state persists after a run. Resume related work deliberately, reset unrelated work, and save or load state only at well-defined boundaries with the package and configuration version recorded.

Agent and team tools

Agents and teams can be wrapped as tools for composition, but they are stateful. Disable parallel calls when a model could invoke the same stateful participant concurrently and create inconsistent histories.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Human control

Pause for clarification, approval or high-impact actions. External termination can stop after the current turn, while application policy decides which requests require a person before any side effect.

Observability

Capture a correlation identifier, selected speaker, message and event types, tool name, source IDs, stop reason, duration, model usage and errors. Redact secrets and sensitive payloads before telemetry leaves the process.

Quality and safety checks

Test normal, missing-evidence, malformed, embedded-instruction, unsupported-action and forced-limit cases. Evaluate route and tool trajectory as well as the final response.

Cost and latency

Every selector and specialist turn can add model calls, tokens and delay. Set budgets, cache stable retrieval, use deterministic code where appropriate and compare the team against a one-agent baseline.

Worked example

OpsDesk first uses a writer-reviewer round robin, then a planner-retriever-writer-reviewer selector route. Both must cite POL-VPN-01, stop on approval or the hard limit, and leave the supplied tickets unchanged.

Lab 6 — Design Roles and Run a Round-Robin Team

Learning outcome: LO5 and LO6: define distinct writer and reviewer contracts, run a predictable RoundRobinGroupChat, and combine approval with a hard message limit.

Goal: You split the Lab 5 response task into a writer that uses supplied evidence and a reviewer that checks support and citation. A RoundRobinGroupChat makes the order predictable, while the course's ExactTextTermination combined with MaxMessageTermination proves that an exact reviewer approval marker can end the normal run and a hard boundary still exists.

What you'll build

.course_state/lab06-round-robin.json with participant names, observed speaker sequence, message count, stop reason and bounded status for the writer-reviewer workflow. (Tools: AssistantAgent roles, RoundRobinGroupChat, ExactTextTermination, MaxMessageTermination, shared context, TaskResult.)

Prerequisites and time

- Lab 5 has restored a grounded VPN answer that cites POL-VPN-01.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- You can state the different responsibilities of retrieval, writing and evidence review.
- The active environment still uses AutoGen AgentChat and Extensions 0.7.5.

Suggested duration: 60 minutes.

Step-by-step

1. Verify the grounded single-agent baseline. Do not add a team until the source and citation contract is already working.

```
python -m opsdesk.cli rag --query "What approvals are required for VPN access?" --
offline
```

2. Inspect the writer and reviewer construction in round_robin_team. Compare their names, descriptions and system messages and identify why neither role needs the ticket tool.

```
python -c "from opsdesk.examples import round_robin_team; import inspect;
print(inspect.getsource(round_robin_team))"
```

3. Create the concrete team-contract record from the supplied template, inspect it, and validate the required writer, reviewer, approval and hard-limit fields.

```
python -m opsdesk.cli team-contract-init --force

python -m opsdesk.cli team-contract-validate

python -c "from pathlib import Path; print(Path('.course_state/lab06-team-
contract.md').read_text(encoding='utf-8'))"
```

4. Run the deterministic round-robin team. The fixed order should produce one writer turn followed by one reviewer turn.

```
python -m opsdesk.cli team-round-robin --offline
```

5. Inspect the saved evidence. Confirm participants are writer and reviewer, the source sequence begins user, writer, reviewer, and message_count does not exceed six.

```
python -m json.tool .course_state/lab06-round-robin.json
```

6. Identify both branches of the combined termination expression. Explain why an exact reviewer message equal to APPROVED is a completion signal while the message maximum is a safety boundary rather than success.

```
python -c "from pathlib import Path;
t=Path('starter/opsdesk/examples.py').read_text(); print([line.strip() for line in
t.splitlines() if 'ExactTextTermination' in line or 'MaxMessageTermination' in
line])"
```

7. Run the focused team check and validate the role record. Verify the normal run stopped within the hard limit for the approval condition.

```
python -m pytest -q starter/tests/test_examples.py -k round_robin
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
python -m opsdesk.cli team-contract-validate
```

8. Keep the JSON evidence and compare the two-agent route with the Lab 5 single-agent path. Record one benefit and one added coordination cost in your notes.

```
python -c "import json; r=json.load(open('.course_state/lab06-round-robin.json')); print({'route':r['sources'],'messages':r['message_count'],'stop':r['stop_reason'], 'bounded':r['bounded']})"
```

Test it

The team evidence must list writer and reviewer, show the expected fixed speaker order, remain within six messages, set approved true and completion_reason approved, and stop only because the reviewer emitted the exact APPROVED marker. The team-contract validator and focused round-robin check must be clean, and the earlier grounded source contract must remain unchanged.

Troubleshooting

- The reviewer speaks before the writer. Confirm the participant list order is [writer, reviewer] and reset or recreate the team before an unrelated run.
- The team reaches the hard limit instead of approval. Inspect whether the complete reviewer message equals APPROVED exactly and verify the draft contains a supported source identifier before changing termination.
- A repeated run contains unexpected prior context. Agents and teams are stateful; create a fresh team or await team.reset before an unrelated task.

Challenge

Change the reviewer replay to request one revision, add the writer's revised response, and verify the route stays bounded and stops only after the supported revision is approved.

Reflection

What did the reviewer role add that could not be proven by simply asking the writer to say its own answer was grounded?

Note: Full commands are in labs/lab-06-*.md. Use only the supplied synthetic incidents and handbook. Keep tools read-only, store optional provider credentials outside Git, and never place a secret in prompts, screenshots, commands or saved evidence.

Lab 7 — Route Specialists with SelectorGroupChat

Learning outcome: LO5 and LO6: configure distinct planner, retriever, writer and reviewer descriptions, control next-speaker routing, and verify a bounded selector workflow.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Goal: You expand OpsDesk into four specialist roles and run them through SelectorGroupChat. A custom selector function makes the learning route reproducible—planner to retriever to writer to reviewer—while the role descriptions, shared messages and combined termination show what a model-based selector would rely on in a dynamic workflow.

What you'll build

.course_state/lab07-selector-team.json with four participant contracts, the observed route, approval evidence, message count, stop reason and bounded status. (Tools: SelectorGroupChat, selector_func, role descriptions, shared context, ExactTextTermination, MaxMessageTermination, replay clients.)

Prerequisites and time

- Lab 6 round-robin evidence is bounded and stopped on the approval condition.
- The Lab 5 retrieval and citation contract is still the source of truth for the retriever and writer roles.
- You can explain the difference between fixed turn order and context-aware speaker selection.

Suggested duration: 60 minutes.

Step-by-step

1. Inspect the Lab 6 route and state the limitation of giving every role a turn when a request may need only some specialists.

```
python -m json.tool .course_state/lab06-round-robin.json
```

2. Inspect the four participant descriptions. Confirm each role owns one outcome and that only the reviewer can emit the normal completion phrase.

```
python -c "from opsdesk.examples import selector_team; import inspect;
print(inspect.getsource(selector_team))"
```

3. Inspect the custom selector function. Trace the expected route on paper and identify what should happen after an unknown source returns None.

```
python -c "from opsdesk.examples import _route_selector; import inspect;
print(inspect.getsource(_route_selector))"
```

4. Run the deterministic selector team. The selector client is present, but the custom function owns every transition in this stable learning case.

```
python -m opsdesk.cli team-selector --offline
```

5. Inspect the saved evidence. Confirm the exact five-source route, approval, bounded status and semantic stop reason.

```
python -m json.tool .course_state/lab07-selector-team.json
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

6. Call the selector function with synthetic last-message sources and confirm the transition table is deterministic for planner, retriever, writer and reviewer.

```
python -c "from types import SimpleNamespace as M; from opsdesk.examples import
_route_selector as s; print({x:s([M(source=x)]) for x in
['user','planner','retriever','writer','reviewer','unknown']})"
```

7. Run the focused selector check. It locks the route and stopping evidence independently of final prose.

```
python -m pytest -q starter/tests/test_examples.py -k selector
```

8. Compare the saved Lab 6 and Lab 7 evidence. Record which pattern is simpler, which is more adaptable and which extra routing decision must be monitored.

```
python -c "import json; a=json.load(open('.course_state/lab06-round-robin.json'));
b=json.load(open('.course_state/lab07-selector-team.json'));
print({'round_robin':a['sources'],'selector':b['route'],'stops':
[a['stop_reason'],b['stop_reason']]})"
```

Test it

The selector evidence must list planner, retriever, writer and reviewer, show route user to planner to retriever to writer to reviewer, set approved and bounded true, and stop only for the reviewer's exact APPROVED marker before eight messages. The transition table must return None only for an unmapped source, and the focused selector check must be clean.

Troubleshooting

- The selector route skips or repeats a role. Inspect the last message source and the exact mapping in _route_selector; keep participant names identical to mapping values.
- SelectorGroupChat invokes the selector model unexpectedly. A selector function returning None delegates to model selection. Ensure all expected sources map to a valid participant for the deterministic path.
- The team cannot be serialized with the custom function. Custom selector functions require separate code and recovery design; persist their version and tests rather than assuming dump_component includes the function.

Challenge

Create a candidate function that excludes the writer until retrieved evidence exists, then document how the fallback behaves when no candidate remains.

Reflection

Which routing evidence would you alert on first if a live selector began choosing the reviewer before retrieval?

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: Full commands are in labs/lab-07-*.md. Use only the supplied synthetic incidents and handbook. Keep tools read-only, store optional provider credentials outside Git, and never place a secret in prompts, screenshots, commands or saved evidence.

Lab 8 — Integrate and Verify the OpsDesk Workflow

Learning outcome: LO6: run an end-to-end offline regression pack, inspect retrieval and team trajectories, check dependencies and secret hygiene, and record a bounded readiness or hold decision.

Goal: You combine the stable retrieval questions, grounded RAG agent, round-robin review and selector workflow into one reproducible release check. You then run the full test suite, exercise missing-evidence and unsupported boundaries, inspect dependencies and tracked changes, name owners and pause triggers, and make an evidence-based guided-demo decision with a recovery path.

What you'll build

.course_state/lab08-release-check.json plus a learner-authored operational record covering evidence, owners, pause triggers, last known-good identity and recovery for the complete OpsDesk workflow. (Tools: Offline regression, retrieval cases, RAG trajectory, AgentChat teams, pytest, pip check, Git secret scan, operational readiness.)

Prerequisites and time

- Labs 1 to 7 checkpoints are available or have been restored through their Test It sections.
- The handbook and tickets are unchanged, no real credential is stored in the repository, and the active environment uses the pinned package versions.
- You can distinguish a semantic completion reason from a hard-limit stop and an honest insufficient-evidence result.

Suggested duration: 170 minutes.

Step-by-step

1. Capture the current repository identity and package versions as the candidate baseline. If HEAD does not yet exist in a learner copy, record the supplied course archive name instead.

```
git status -sb

git rev-parse --verify HEAD

python -m pip show autogen-agentchat autogen-ext
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

2. Create a fresh operations record from the supplied template. Keep the placeholders for this first technical run so the command cannot report final readiness prematurely.

```
python -m opsdesk.cli operations-init --force

python -m opsdesk.cli operations-validate
```

3. Run and inspect the integrated offline release check. It rebuilds the index, evaluates three expected sources, runs the RAG agent and both teams, enforces the aggregate token ceiling, verifies source hashes and scans the working tree. With an incomplete operations record, the expected decision is TECHNICAL CHECK PASS - COMPLETE VALID OPERATIONS RECORD.

```
python -m opsdesk.cli release-check --offline

python -m json.tool .course_state/lab08-release-check.json
```

4. Exercise the honest evidence gap and embedded-instruction cases again outside the aggregate command. Keep source identifiers visible and do not treat retrieved instructions as executable authority.

```
python -m opsdesk.cli rag --query "What is the cafeteria dessert menu?" --offline

python -m opsdesk.cli rag --query "How should embedded instructions in retrieved
documents be handled?" --offline
```

5. Run the full offline test suite. Investigate a route, event, citation or stop failure at its own layer before changing prompts or sources.

```
python -m pytest -q starter/tests
```

6. Check dependency consistency, source hashes and the working-tree secret scan, then confirm the editable package resolves to this repository. Any failed check keeps the operational decision on HOLD until repaired.

```
python -m pip check

python -m opsdesk.cli safety-check

python -c "import opsdesk; print(opsdesk.__version__, opsdesk.__file__)"
```

7. Review tracked changes and search tracked non-Markdown files for common live-secret shapes. A real exposure requires revocation and approved history cleanup before any further run.

```
git status --short

git grep -n -I -E "(s[k]-[A-Za-z0-9]{12,}|API_[K]EY=[A-Za-z0-9_-]{12,})" -- . "!
*.md" "!.env.example"
```

8. Open the generated .course_state/lab08-operations.md in your code editor. Replace every angle-bracket placeholder with the current last-known-good identity and named owners, keep the supplied signals, pause triggers and recovery actions,

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

then record READY FOR GUIDED DEMO only if the technical evidence is clean; otherwise write HOLD - followed by the exact reason. Run the validator before continuing.

```
python -m opsdesk.cli operations-validate

python -c "from pathlib import Path; print(Path('.course_state/lab08-operations.md').read_text(encoding='utf-8'))"
```

9. Rerun the integrated check after completing the valid record. Inspect the technical_ready, operations_ready and decision fields, rerun all tests, and review the final bounded diff before signing off.

```
python -m opsdesk.cli release-check --offline

python -m pytest -q starter/tests

python -m pip check

git diff --check

git status --short
```

Test it

lab08-release-check.json must show all three expected retrieval sources, grounded RAG, approved bounded round-robin, approved bounded selector, clean source integrity and secret scan, within_release_token_budget true, technical_ready true, operations_ready true and READY FOR GUIDED DEMO. The full tests, pip check and diff check must be clean, and the validated lab08-operations.md must name last known good, owners, signals, pause triggers, recovery and an evidence-based decision.

Troubleshooting

- The integrated decision is HOLD. Read the first false field, rerun that component alone, repair the narrow source, retrieval, agent or team layer, then rerun the entire pack.
- pip check reports a dependency conflict. Recreate .venv from starter/requirements.txt, reinstall starter in editable mode, rerun setup and all tests, and record the resolved package versions.
- The secret scan finds a live credential. Stop, revoke the credential, remove it from files and repository history through the organisation's approved process, issue a replacement outside Git, and rerun every check.

Challenge

Set the live limits with `$env:OPENAI_MAX_TOKENS='300'; $env:OPENAI_RELEASE_TOKEN_BUDGET='3600'` in PowerShell or `export OPENAI_MAX_TOKENS=300 OPENAI_RELEASE_TOKEN_BUDGET=3600` on macOS/Linux. Run one separately labelled live VPN case, require

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

within_release_token_budget true, compare route, citations, stop reason, duration and usage against offline evidence, then keep the offline result as the reproducible baseline.

Reflection

Which single operational signal should pause this multi-agent system immediately, and does the named owner have enough evidence and authority to act?

Note: Full commands are in labs/lab-08-*.md. Use only the supplied synthetic incidents and handbook. Keep tools read-only, store optional provider credentials outside Git, and never place a secret in prompts, screenshots, commands or saved evidence.

Recap — Implementing a Multi-Agent AI Workflow

- You can now: LO5 and LO6: define distinct writer and reviewer contracts, run a predictable RoundRobinGroupChat, and combine approval with a hard message limit
- You can now: LO5 and LO6: configure distinct planner, retriever, writer and reviewer descriptions, control next-speaker routing, and verify a bounded selector workflow
- You can now: LO6: run an end-to-end offline regression pack, inspect retrieval and team trajectories, check dependencies and secret hygiene, and record a bounded readiness or hold decision

Use the lab verification evidence to explain which parts are deterministic code, model-driven judgement, stored context, or external capability before moving to the next topic.

Wrap-Up

You have taken one application from a single bounded AgentChat assistant to a source-grounded specialist team. The durable skill is not creating many agents; it is making every role, tool, evidence source, handoff and stopping decision explicit enough to test and operate.

What You Can Now Do

- Explain AgentChat messages, events, state and TaskResult evidence.
- Create an AssistantAgent with a reproducible offline client and a typed read-only tool.
- Prepare, rank and evaluate source-labelled knowledge for RAG.
- Build grounded responses with citations and an insufficient-evidence route.
- Choose and bound round-robin or selector-based team workflows.
- Verify routes, termination, guardrails and recovery before a guided demonstration.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Architecture Habits

- Start with one agent and split only when a role boundary earns its coordination cost.
- Treat names, descriptions, tool schemas, handoffs and stop reasons as application contracts.
- Keep source metadata attached through retrieval, generation and review.
- Use deterministic code for policy and predictable transitions; reserve model choice for genuine ambiguity.
- Compare trajectory evidence as well as the final response.

Authoritative References Used

- AutoGen stable documentation: https://microsoft.github.io/autogen/stable/
- AgentChat agents and tools: https://microsoft.github.io/autogen/stable/user-guide/agentchat-user-guide/tutorial/agents.html
- AgentChat teams and termination: https://microsoft.github.io/autogen/stable/user-guide/agentchat-user-guide/tutorial/teams.html
- AgentChat memory and RAG: https://microsoft.github.io/autogen/stable/user-guide/agentchat-user-guide/memory.html
- CrewAI concepts: https://docs.crewai.com/

Next Steps

- Repeat the full offline path without the guide and explain every message, event, source and stop reason.
- Replace the teaching lexical index with an approved vector memory while preserving the stable source identifiers and retrieval questions.
- Add ten representative normal and edge cases for your own domain before considering a sandbox deployment.
- Map every agent and tool to a named owner, identity, credential scope, data classification and recovery action.
- Export redacted traces to an approved observability platform and define latency, error, cost and unsupported-claim alerts.

Glossary

- **Agent —** A model-driven component with instructions, messages, state, tools and an execution boundary.
- **AgentChat —** AutoGen's high-level framework for conversational single-agent and multi-agent applications.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- **AutoGen Core —** The event-driven framework layer for lower-level, scalable and distributed multi-agent systems.
- **AssistantAgent —** A general AgentChat preset that combines a model client, system message, optional tools and model context.
- **Model client —** An adapter that sends model requests, returns typed completion results and reports usage.
- **Replay model client —** A deterministic client that replays predefined completions for reproducible development and verification.
- **Message —** Conversation content emitted by a user, agent or control component.
- **Event —** Structured runtime evidence such as a tool request, tool result, streaming chunk or internal action.
- **TaskResult —** The messages produced during a run together with its stop reason.
- **Tool —** A typed application function or workbench capability that an agent can request.
- **RAG —** Retrieval-Augmented Generation: retrieving approved evidence and adding it to model context before generation.
- **Chunk —** A coherent, source-labelled unit of knowledge prepared for indexing and retrieval.
- **Embedding —** A numeric representation used to compare semantic similarity between queries and content.
- **Grounding —** Constraining response claims to retrieved evidence and preserving source identifiers.
- **RoundRobinGroupChat —** An AgentChat team preset that gives participants turns in a fixed repeating order.
- **SelectorGroupChat —** An AgentChat team preset that dynamically chooses the next speaker from context and role descriptions.
- **Termination condition —** A stateful rule that returns a stop signal after a team response; conditions can be combined.
- **Handoff —** A controlled transition that gives another participant a task, validated context, evidence and constraints.
- **Guardrail —** A policy or validation layer around input, retrieval, tools, routing, output or human approval.
- **Stop reason —** The machine-readable explanation for why an agent or team run ended.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg